The AI Factory

Jensen Huang, NVIDIA, and the Revolution That Changed Everything

Based on Lex Fridman Podcast #494

Lex Fridman Podcast #494 with Jensen Huang

About This Book

This book was created by transforming a long-form conversation between Lex Fridman and Jensen Huang (CEO of NVIDIA) into narrative prose. The source material is Lex Fridman Podcast episode #494, published March 2026.

Every fact, figure, quote, and insight in this book comes directly from that conversation. No outside research has been added. The goal is to preserve the depth and richness of the original discussion while presenting it in a form that reads like a book, not a transcript.

Tone: Bill Bryson meets Neil deGrasse Tyson. Warm, witty, curious, vivid, and accessible. Technical ideas explained in plain language.

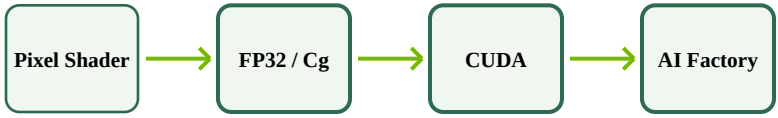
Contents

CHAPTER 1

The Bet That Nearly Killed Them

How a decision to burn through every dollar of profit built the foundation for a four-trillion-dollar empire.

NVIDIA Evolution: From Accelerator to AI Factory



Step-by-step expansion from narrow accelerator to universal computing platform.

* * *

There is a particular flavor of corporate gamble that looks, in retrospect, like genius, but in the moment feels a lot more like jumping off a cliff while still sewing the parachute. The decision by NVIDIA to embed CUDA into every GeForce graphics card it shipped was exactly that kind of gamble. It nearly destroyed the company. It also, as things turned out, laid the foundation for the most consequential computing platform of the twenty-first century.

To understand why the bet mattered, you have to understand what NVIDIA was before it made the leap. In the early 2000s, NVIDIA was an accelerator company, and a successful one. Its GeForce GPUs had

become the gold standard for PC gaming, selling millions of units per year. The business was healthy. The margins were solid, if unspectacular, hovering around 35 percent. The company's market capitalization sat somewhere in the neighborhood of six to eight billion dollars. For a chip company focused on a single application domain, this was a comfortable, even enviable, position.

But Jensen Huang, NVIDIA's co-founder and CEO, had a problem with comfort. He could see, with the clarity of an engineer who thinks in systems rather than products, that accelerators faced a structural ceiling. The more specialized a chip became, the narrower its market. The narrower the market, the smaller the R&D; budget. And the smaller the R&D; budget, the less influence the company could exert on the broader world of computing. It was a trap disguised as a niche.

* * *

The escape route ran through programmability. NVIDIA had already taken small, deliberate steps in that direction. The programmable pixel shader was the first: a way to let developers write custom instructions for the GPU rather than relying on fixed-function hardware. Then came IEEE-compatible floating-point arithmetic (FP32), which was a quiet earthquake. Suddenly, researchers working on stream processors and data-flow architectures perked up. A GPU that could do real math, compliant with the same standards as a CPU, was no longer just a gaming chip. It was a potential computing engine.

From FP32 came Cg, a shading language. From Cg came, eventually, CUDA: a full programming model that let developers write general-purpose code to run on NVIDIA's massively parallel hardware. Each step was a small expansion of the aperture, carefully balanced between the specialization that made NVIDIA's chips fast and the generalization that could make them universal. Jensen described the

tension this way: the better NVIDIA became as a computing company, the worse it risked becoming as a specialist. And the more it specialized, the less capacity it had to do broad computing. The company had to walk an extraordinarily narrow path, and it had to do so step by step, never surrendering one advantage in pursuit of the other.

CUDA was the biggest step yet. But inventing it was only half the problem. The other half, the half that almost sank the ship, was getting developers to actually use it.

* * *

Here is the thing about computing platforms that Jensen understood with bone-deep conviction, the kind of conviction that comes from watching decades of tech history: install base is everything. Not elegance. Not raw performance. Not even the quality of the underlying architecture. Install base. He pointed to the x86 instruction set as proof. No architecture in the history of computing had attracted more criticism for its lack of elegance. Meanwhile, beautifully designed RISC architectures, built by some of the brightest computer scientists alive, had largely failed. The reason was brutally simple. x86 was everywhere. Developers write for the platforms that reach the most people. Reach is what matters. Everything else is secondary.

So the question became: how do you build an install base for a brand-new computing architecture when nobody has heard of it yet and nobody is asking for it?

Jensen's answer was GeForce. NVIDIA was already shipping millions of GeForce GPUs into PCs every year. If the company embedded CUDA into every single one of those chips, whether gamers wanted it or not, whether they would ever use it or not, then overnight CUDA would have a massive install base. Every researcher in a university lab, every scientist who happened to be a gamer, every

student who built their own PC from off-the-shelf components would have a supercomputer hiding inside their graphics card. All NVIDIA had to do was go out and teach them how to use it.

There was just one catch. Embedding CUDA into GeForce increased the manufacturing cost of each chip by roughly 50 percent. And gamers, who were the people actually buying GeForce cards, would not pay a penny more for capabilities they did not understand and would never touch. The cost increase would come straight out of NVIDIA's margins. For a company running at 35 percent gross margin, that was not a haircut. It was an amputation.

* * *

The consequences arrived swiftly. NVIDIA's market capitalization, which had been sitting in the six-to-eight-billion-dollar range, collapsed to roughly one and a half billion. The company sat in that valley for a long time, clawing its way back while carrying the full burden of CUDA on every consumer chip it shipped.

Jensen had to make the case to his board and his management team, and the case was not easy to make. He could reason through it: someday CUDA might go into workstations and supercomputers, where higher margins could absorb the cost. But "someday" is a word that does not pay the bills. The financial suffering was immediate. The payoff was theoretical and, at best, a decade away.

What sustained Jensen through that decade was a particular mode of conviction. He described it as a reasoning process that, at some point, becomes so clear and so internally consistent that the future it predicts feels inevitable. Not probable. Inevitable. The suffering along the way is real, but the destination is not in question. You manifest a future in your mind, and then you engineer your way toward it, one brick at a time.

This was not, he insisted, the kind of leadership where a CEO disappears into a back room, emerges with a manifesto, announces a new mission statement, fires half the company, and rebrands with a new logo. Jensen's method was almost the opposite. When he began to see something important, he did not keep it to himself. He started talking about it immediately, to everyone near him: his direct reports, the management team, the board, customers, partners. He would take every new piece of evidence (a research paper, a customer insight, an engineering milestone) and use it to nudge the collective belief system of everyone around him, brick by brick, day by day.

By the time he was ready to make a formal declaration, the declaration felt obvious. He wanted people to react not with shock, but with a mild impatience, as if to say: what took you so long?

* * *

That is, in fact, almost exactly what happened when NVIDIA went all in on deep learning years later. Jensen had been seeding the idea across the company for so long that, on the day he announced it, the buy-in was essentially unanimous. The same pattern repeated with the acquisition of Mellanox, with the pivot to data-center-scale computing, and with each successive generation of hardware designed for an AI workload that was growing faster than anyone outside NVIDIA fully appreciated.

But all of those later triumphs traced their roots back to the original, painful bet. CUDA on GeForce. A consumer product subsidizing an invisible computing platform that nobody had asked for and that, for years, nobody seemed to want.

Jensen liked to say that NVIDIA was the house that GeForce built. Not because GeForce was NVIDIA's most profitable product (it was not, not by a long shot). But because GeForce was the vehicle that carried CUDA into the world. Researchers and scientists discovered it

because they were gamers. University labs built GPU clusters from PC components. And when deep learning arrived, when the neural networks that would reshape every industry on Earth needed massively parallel hardware to train on, CUDA was already there, already installed, already familiar.

The bet that nearly killed NVIDIA turned out to be the decision that made everything else possible. The market capitalization that had collapsed to a billion and a half dollars would, in the fullness of time, rise past four trillion. Jensen's parachute, it turned out, had been sewn just in time.

CHAPTER 2

Sixty Reports and No One-on-Ones

Inside the radical management system that turned NVIDIA into the world's most complex co-design machine.

* * *

Most CEOs of large technology companies have somewhere between five and twelve direct reports. The number is not arbitrary. It reflects a deeply entrenched assumption about how organizations should work: a leader can only meaningfully manage a handful of people, and those people manage a handful more, and so on, layered downward in a tidy pyramid until you reach the people who actually build things. Jensen Huang's direct staff is more than sixty people. Sometimes more.

He does not hold one-on-one meetings with them. He cannot, not in any conventional sense. Sixty individual check-ins would consume every waking hour and leave no time for the actual work of running a company. But the reason Jensen structured NVIDIA this way had nothing to do with ego or some fetish for flat hierarchies. It had everything to do with the nature of the product NVIDIA was trying to build.

* * *

When Jensen looked at other companies' organizational charts, he saw a sameness that baffled him. The hamburger org chart. The soft org chart. The car company org chart. They all looked the same, regardless of what the company made or the environment in which it operated. This struck him as deeply illogical. A company, in Jensen's view, is a machine. Its purpose is to produce an output. The architecture of that machine should be dictated by the output it needs to create and the environment in which it must operate. If those things change, the machine should change too.

NVIDIA's output, by the mid-2020s, was no longer a single chip. It was an entire computing system of staggering complexity: GPUs, CPUs, networking chips, scale-up switches, scale-out switches, high-bandwidth memory, power delivery systems, cooling infrastructure, and the software that orchestrated all of it. A single NVLink-72 rack contained 1.3 million components, roughly 1,300 chips, and required 200 suppliers to manufacture. The Vera Rubin pod (announced at GTC) packed seven chip types, five purpose-built rack types, 40 racks, 1.2 quadrillion transistors, nearly 20,000 NVIDIA dies, over 1,100 Rubin GPUs, 60 exaflops of compute, and 10 petabytes per second of scale bandwidth into a single deployable unit. And NVIDIA would need to produce roughly 200 of those pods per week.

Building a system like that requires what Jensen calls "extreme co-design," the simultaneous optimization of every layer in the stack: algorithms, applications, system software, chips, networking, power, and cooling. No single discipline can be designed in isolation. A decision about cooling affects power delivery, which affects chip design, which affects networking, which affects the software that distributes workloads across the system. Everything touches everything.

And that is why Jensen's staff is so large, and why he refuses to hold one-on-one meetings.

* * *

Among those sixty-plus direct reports are world-class specialists in memory, CPUs, GPUs, optical networking, algorithms, architecture, chip design, and system software. Almost all of them have deep engineering roots. When a problem needs to be discussed, say, the thermal constraints on a new rack design, Jensen does not call in the cooling expert for a private chat. He convenes the group. The cooling expert presents the problem. The memory specialist listens in. So does the networking lead, the power delivery architect, the system software engineer, and anyone else whose domain might be affected.

The result is a standing conversation that never really ends. Problems are presented, and the entire team attacks them collectively. If you are a specialist in one area and a discussion moves to another, you are free to tune out. But if a discussion touches your domain and you fail to contribute, Jensen will call you out. The expectation is not that everyone speaks all the time. It is that everyone pays attention when it matters, and that no critical interdependency slips through the cracks because some specialist was off in a one-on-one meeting when the real decision was being made.

This is, in essence, the organizational expression of extreme co-design. The company mirrors the product. Just as every component in the rack must be optimized in concert with every other component, every discipline in the company must be aware of what every other discipline is doing. The architecture of the company reflects the architecture of the machine it builds.

* * *

Jensen traced NVIDIA's evolution through a series of inflection points, each one a deliberate expansion of the company's aperture. It began as a narrowly focused accelerator company, building GPUs for gaming. The programmable pixel shader was the first step toward generality.

IEEE-compatible FP32 was the second. Cg and then CUDA were the third and fourth. Each step introduced a tension between specialization (which made the chips fast) and generalization (which made them useful for more things). The company had to find the narrow path that expanded its reach without sacrificing its edge.

The environment changed too. When deep learning arrived, NVIDIA recognized it before almost anyone else and went all in. But the shift to AI computing demanded more than just better GPUs. It demanded an entirely new way of thinking about what a computer was. The "unit of computing," as Jensen described it, had evolved. First it was a GPU. Then it was a whole computer. Then a cluster. Now it was an AI factory: a gigawatt-scale installation connected to the power grid, staffed by thousands of engineers, and producing not files or web pages but tokens, the raw currency of artificial intelligence.

That evolution in the product demanded a corresponding evolution in the company. Jensen did not reorganize NVIDIA with layoffs and new mission statements. He reorganized it the same way he approached every other change: gradually, transparently, by shaping belief systems one conversation at a time. By the time any formal shift was announced, the people affected already understood why it was necessary. They had been living inside the reasoning for months or years.

* * *

The method had a name, or at least a signature: the "what took you so long?" principle. Jensen's goal, whenever he needed the company to change direction, was to prepare the ground so thoroughly that his announcement felt late rather than early. He wanted employees, board members, customers, and partners to react with mild impatience rather than shock.

He achieved this by doing something that many leaders consider risky: he talked openly and constantly about what he was thinking. When a new insight struck him, he did not retreat to study it privately. He shared it with whoever was near. As evidence accumulated, as new papers were published, as engineering milestones were hit, he would use each one as an opportunity to nudge the collective understanding of everyone around him. Over weeks and months, the bricks accumulated into a wall, and by the time he pointed to the wall and said "let's go," the response was a collective shrug. Of course. Obviously. What took you so long?

This technique extended well beyond NVIDIA's own employees. Jensen used GTC, the company's annual technology conference, as a tool for shaping the belief systems of the entire industry. Partners, customers, cloud providers, and competitors all attended. The keynotes were not just product launches. They were carefully structured arguments about the future, each one building on the last, each one laying another brick. When Jensen talked about agentic AI systems at GTC two years before they became a mainstream reality, he was not making a prediction. He was planting a seed that he intended to water, in public, until the harvest was so obviously ripe that no one would question the timing.

* * *

The sixty-report structure, the absence of one-on-ones, the relentless transparency, and the slow-burn consensus-building were not separate tactics. They were aspects of a single philosophy: the company should be a system that produces its product, and the system should be designed with the same rigor and intentionality as the product itself. Most organizations inherit their structure from convention. Jensen designed his from first principles.

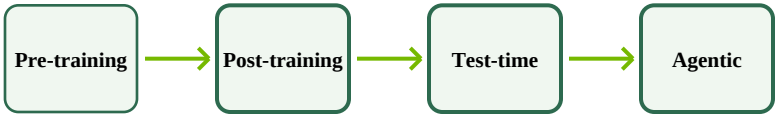
It was, in its own way, an act of extreme co-design.

CHAPTER 3

The Four Laws of Intelligence

Why every supposed wall in AI turned out to be a door, and why the only real limit is compute.

The Four Scaling Laws of AI Intelligence



Each law feeds the next in a continuous flywheel, with agentic data looping back to pre-training.

* * *

In the short history of modern artificial intelligence, there has been no shortage of prophets announcing the end. Each time, the prophecy has followed the same arc: a scaling law runs into a wall, the industry panics, commentators declare that AI has hit its ceiling, and then, within a year or two, someone discovers a new scaling law that renders the panic quaint.

Jensen Huang watched each cycle with a mixture of amusement and strategic clarity. He was not surprised that walls kept appearing. He was not even surprised when they fell. What interested him was the pattern,

and what the pattern meant for the future of computing.

* * *

The first scaling law was pre-training: the discovery, formalized most clearly by researchers at OpenAI in the late 2010s, that larger models trained on correspondingly larger datasets became reliably smarter. More parameters plus more data equaled more capability. The relationship was not just empirical; it was almost eerily predictable, following smooth curves on logarithmic plots. For a while, this was the only scaling law that mattered. Build bigger models. Feed them more data. Watch them improve.

The panic arrived when people began to notice that the supply of high-quality human-generated data was not infinite. Ilya Sutskever, one of the architects of modern deep learning, made a remark to the effect that pre-training, in its current form, was reaching its limits. The data was running out. The industry interpreted this as a death sentence.

Jensen's response was characteristically unsentimental. Of course the data was finite, if you only counted the data that humans had written down in its raw, original form. But what people had forgotten, or perhaps never fully appreciated, was that most of the data humans use to teach and inform each other is, in a meaningful sense, synthetic. It did not emerge from nature. Someone created it. Someone else consumed it, modified it, augmented it, and regenerated it. Human knowledge has always been a chain of transformations applied to a relatively small seed of ground truth.

AI, Jensen argued, had simply reached the point where it could do the same thing. Take a kernel of real-world data. Augment it. Enhance it. Synthetically generate vast quantities of new training material grounded in that kernel. The proportion of training data that was human-generated would shrink, but the total volume of useful data

would keep growing. The bottleneck was no longer data. It was compute. If you had enough compute to generate the synthetic data, and enough compute to train on it, intelligence would keep scaling.

* * *

The second scaling law was post-training: the process of refining a pre-trained model through fine-tuning, reinforcement learning from human feedback, and other techniques that sharpened its capabilities for specific tasks. This was, in some ways, the less glamorous sibling of pre-training, but it mattered enormously. Post-training was where raw pattern recognition became useful skill.

The third scaling law was test-time compute, and this was the one that caught most of the industry off guard. For years, the conventional wisdom had been that inference (the process of actually running a trained model to generate answers) was the easy part. Pre-training was hard because it required enormous clusters running for months. Inference, the thinking went, could be done on small, cheap, specialized chips. The future of inference would be commoditized. Anyone could build an inference chip. NVIDIA's expensive, complex GPUs were overkill for the task.

Jensen had always found this logic baffling. Inference is thinking. Pre-training is reading. And thinking, by any reasonable analysis, is harder than reading. Pre-training involves memorization and pattern recognition: scanning vast corpora, identifying statistical relationships, and encoding them into the model's weights. Inference, especially as models become more capable, involves reasoning, planning, decomposition, search, and exploration. A model that needs to solve a genuinely novel problem cannot simply retrieve a memorized answer. It must break the problem into sub-problems, reason about each one from first principles or from prior experience, explore multiple paths, and

synthesize a response. That process is computationally intense, and it becomes more intense as the problems become harder.

Test-time scaling proved Jensen right. The more compute you threw at inference (letting the model "think longer" by generating and evaluating more intermediate steps), the better the results became. Inference was not a commodity workload. It was, potentially, the largest and most demanding workload in all of computing.

* * *

The fourth scaling law was agentic: the discovery that intelligence could be multiplied not just by making individual models smarter, but by letting them spawn and coordinate teams of sub-agents. A single AI agent, equipped with a large language model and the ability to use tools, could accomplish a great deal. But a system that could spin off a dozen sub-agents, each tackling a different aspect of a complex problem, could accomplish vastly more. It was, Jensen observed, much easier to scale a company by hiring more employees than by making any single employee infinitely smarter. The same principle applied to AI.

Agentic systems did not just consume compute. They generated data. As agents worked, they produced logs of their actions, their successes, their failures, and their discoveries. Some of that data would prove valuable enough to memorize, feeding back into pre-training. The refined insights would flow into post-training. Enhanced capabilities would improve test-time reasoning. And better reasoning would make the agents more effective, generating still more data.

The four scaling laws were not independent staircases. They were a flywheel, each one feeding the others in a continuous cycle.

* * *

For NVIDIA, the implications were profound and, to Jensen, almost mathematically obvious. Intelligence was going to scale. The only constraint was compute. And compute was what NVIDIA built. Every supposed wall in AI, every moment when the industry believed the party was over, had turned out to be a temporary pause before the discovery of a new dimension in which to grow. The direction of growth might change (from data-limited to compute-limited, from pre-training to inference, from single models to swarms of agents), but the need for more compute only ever went up.

Jensen described the transition in a vivid metaphor. Computing had once been a retrieval system: humans pre-wrote content, stored it in files, and computers fetched the right file when asked. The entire architecture of the traditional data center was optimized for storage and retrieval. In the AI era, computing had become generative. Machines did not fetch pre-recorded answers; they generated new, contextually relevant responses in real time. That required vastly more processing and fundamentally different hardware.

The old data center was a warehouse. The new data center was a factory. And factories, unlike warehouses, directly correlate with revenue. The tokens they produced (the raw units of AI output) were not just technically interesting. They were economically valuable, segmenting into tiers the way consumer products do: free tokens, premium tokens, and everything in between. The idea that someone would willingly pay a thousand dollars per million tokens for sufficiently specialized intelligence was, in Jensen's estimation, not a question of if but only of when.

* * *

The flywheel was spinning. The four laws were feeding each other. And at the center of it all, inexorably, sat the need for compute, compute,

compute. Every doubt that had been raised about the limits of AI had been answered not by accepting the limit, but by discovering a new axis of scaling that made the old limit irrelevant.

Jensen did not see this as luck. He saw it as the predictable consequence of a technology that was fundamentally sound and a demand that was fundamentally real. The only question was whether the world could build the factories fast enough.

CHAPTER 4

The Supply Chain Whisperer

How Jensen Huang persuaded an entire industrial ecosystem to bet billions on a future only he could see.

The NVIDIA Supply Chain Ecosystem

200 suppliers, 1.3M components per rack, connected by trust and shared vision.

* * *

In the spring of 2023, Jensen Huang sat down with the CEOs of several of the world's largest memory manufacturers and made a pitch that, by any conventional standard, sounded absurd. High-bandwidth memory (HBM), a technology used at the time almost exclusively in niche supercomputers, was about to become a mainstream product for data centers. The volumes would be enormous. The companies that invested early would reap record-setting revenues. The companies that hesitated would be left behind.

The reaction was predictable. HBM was a specialty product. Data centers ran on DDR memory. The idea that this exotic, expensive,

low-volume technology would suddenly become a mass-market commodity required believing that something fundamental had changed about the nature of computing itself. Some of the CEOs believed Jensen. Others were skeptical. But the ones who believed and invested found themselves, two years later, posting the best financial results in their companies' 45-year histories.

Jensen made an even stranger pitch around the same time: take the low-power LPDDR5 memory used in cell phones and adapt it for use in data-center supercomputers. Cell phone memory in supercomputers? The idea was counterintuitive to the point of seeming comical. But Jensen explained the reasoning, and the companies that listened discovered a new market that had not existed before.

* * *

This was not a one-off event. It was a pattern, and it was central to how NVIDIA operated. Jensen did not simply design and build computing systems. He designed and built the supply chain that would produce them.

The numbers made the stakes clear. A single Vera Rubin rack contained roughly 1.3 to 1.5 million components, sourced from approximately 200 suppliers. The entire NVLink-72 system was so dense and complex that it could no longer be assembled inside a data center the way previous generations had been. Instead, complete supercomputers had to be manufactured and tested in the supply chain itself, then shipped as finished units weighing two to three tons apiece. This shift had cascading consequences. If the target was 50 gigawatts of deployed supercomputing capacity, and the manufacturing cycle was one week per batch, then the supply chain itself needed a gigawatt of power just to build and test the systems before they shipped. The supply chain had to become, in effect, a collection of temporary data centers.

Jensen's job, in large part, was to make sure that every link in this chain understood what was coming and had time to invest accordingly. He flew to suppliers personally. He explained the dynamics of the AI market, the growth drivers, and the timeline. He drew pictures and reasoned from first principles. He asked each partner to make capital investments of several billion dollars. And he did this not by issuing ultimatums, but by giving people every opportunity to question him, to push back, to challenge the logic. By the time the conversation was over, the suppliers typically knew what to do, not because they had been told, but because they had been persuaded.

* * *

The relationship with TSMC, the Taiwanese semiconductor manufacturer that fabricated NVIDIA's most advanced chips, was the most critical of all, and it rested on something unusual for a business partnership of its scale: there was no contract.

Jensen and TSMC had been working together for three decades. Hundreds of billions of dollars of business had flowed between the two companies, all of it governed not by legal agreements but by trust. Jensen described TSMC with a kind of reverence. Their technology was extraordinary, yes, but that was not the deepest source of their strength. What made TSMC remarkable was their ability to orchestrate the dynamic demands of hundreds of companies simultaneously: production schedules shifting, volumes rising and falling, emergency wafer starts, customers pulling orders forward or pushing them out. Somehow, through all that chaos, TSMC ran its factories with high throughput, high yields, excellent costs, and impeccable delivery. When wafers were promised to arrive, they arrived. TSMC understood that they were not just manufacturing silicon. They were helping their customers run their businesses.

Then there was the culture. TSMC had achieved something that most organizations find impossible: being simultaneously world-class at technology and world-class at customer service. Plenty of companies excelled at one or the other, but TSMC managed to push the absolute frontier of semiconductor physics while also treating every customer interaction as a sacred obligation. It was this combination, the technology, the operational excellence, and the culture of service, that had created an intangible asset more valuable than any transistor: trust.

Jensen put it starkly. He trusted TSMC enough to build his entire company on top of theirs. That was not a small thing. NVIDIA did not own a single fabrication facility. Every chip it designed, from the most advanced data-center GPU to the most humble automotive processor, was manufactured by TSMC. If that relationship failed, NVIDIA's business would collapse overnight. The fact that it had endured for thirty years without a contract was, Jensen believed, not a sign of recklessness. It was the strongest possible evidence that the trust was real.

* * *

There was one more story about TSMC that Jensen confirmed, a story that had circulated in the semiconductor industry for years. In 2013, Morris Chang, the legendary founder of TSMC, offered Jensen the chance to become TSMC's chief executive.

Jensen did not dismiss the offer. He was deeply honored. Morris Chang was one of the most consequential figures in the history of technology, and TSMC was, by any measure, one of the most important companies ever created. But Jensen had seen, in his mind's eye, what NVIDIA was going to become. The work was too important and the responsibility too personal to hand off. He declined, not because the offer was inadequate, but because the mission he was already on could not be abandoned.

Years later, with NVIDIA's market capitalization exceeding TSMC's, Jensen reflected on the decision without regret. He had chosen to stay, and by staying, he had positioned himself to help both companies. NVIDIA and TSMC, in Jensen's telling, were two of the greatest engineering organizations in history. Their partnership, built on trust and sustained by mutual respect, was one of the load-bearing pillars of the entire AI revolution.

* * *

The supply chain story was, in the end, an extension of the same philosophy Jensen applied inside NVIDIA. He did not hoard information. He did not wait for the perfect moment to reveal his plans. He told people what he was thinking, explained his reasoning, invited challenges, and gave his partners enough time and context to make their own informed decisions. The result was not a supply chain that followed orders. It was a supply chain that shared a vision.

When asked whether the bottlenecks in the supply chain (ASML's EUV lithography machines, TSMC's advanced packaging, SK Hynix's high-bandwidth memory) kept him up at night, Jensen gave an answer that was almost Zen-like in its simplicity. He had told his partners what he needed. They had understood. They had told him what they were going to do. And he believed them.

He could go to sleep.

CHAPTER 5

Power, the Invisible Bottleneck

The untold story of the 40 percent of the grid that sits idle, and why AI data centers might be the key to unlocking it.

* * *

Of all the obstacles that might slow the AI revolution, the most prosaic is also, perhaps, the most consequential: electricity. Not the theoretical availability of energy (humanity generates enormous amounts of it) but the practical availability of power at the specific locations, in the specific quantities, and with the specific guarantees that modern data centers require. It is a problem that sounds simple and is, in practice, maddeningly complex, tangled in physics, economics, regulation, and a web of contractual obligations that most people, including most technology CEOs, have never thought about.

Jensen Huang had thought about it quite a lot.

* * *

The starting point was a number that, once you heard it, was difficult to stop thinking about. The electrical grid in most developed countries is designed for the worst-case condition, plus a margin of safety. The worst case is a handful of extreme-weather days each year: the coldest days in winter, the hottest days in summer. On those days, the grid must

deliver maximum power to hospitals, airports, transportation systems, residential heating and cooling, and every other piece of critical infrastructure. The grid must be built to handle that peak, and it must have some headroom above it for safety.

But 99 percent of the time, the grid is nowhere near peak capacity. On an average day, utilization hovers around 60 percent. The remaining 40 percent is simply there, powered up, ready to go, sitting idle. It has to be there, because when those handful of extreme days arrive, lives depend on it. But on every other day, it is wasted capacity: generators spinning, transformers humming, transmission lines energized, all producing power that nobody is using.

Jensen's proposal was elegant in its simplicity. What if data centers could use that idle capacity? Not claim it permanently, not add to the grid's peak demands, but absorb the excess on the 99 percent of days when it was going to waste. And on the rare occasions when the grid needed to reclaim that power for hospitals and airports and homes, the data centers would simply step back. They would run slower. They would shift workloads to other locations. They would degrade gracefully, accepting slightly longer response times in exchange for giving the grid the headroom it needed.

* * *

The engineering challenge was real but, in Jensen's assessment, entirely tractable. Data centers could be designed to operate across a range of power levels, dynamically adjusting their compute intensity based on signals from the utility. Critical workloads (those that could not tolerate any interruption) would be routed to data centers with guaranteed full-time power. Everything else would run on the flexible tier, absorbing excess capacity when available and throttling down when asked.

The obstacles were not primarily technical. They were institutional. Three parties were involved, and all three had to change their behavior.

The first was the end customer: the companies and individuals purchasing compute from cloud service providers. These customers demanded perfection. Their contracts specified 99.9999 percent uptime, "six nines" in industry jargon. They wanted guarantees that their workloads would never be interrupted, under any circumstances, for any reason. The customers had good reasons for this (downtime was expensive), but the cumulative effect of all those ironclad contracts was to force cloud providers into a rigid posture that made flexible power consumption impossible.

The second was the cloud service providers themselves. Their operations teams, often operating independently of the CEO, negotiated contracts that locked in the six-nines requirement and passed it downstream to the utilities. Jensen suspected that most CEOs of major cloud companies were not paying close attention to the terms of these power contracts. He intended to bring it to their attention directly.

The third was the utilities themselves. Power companies were accustomed to offering a single tier of service: highly reliable, always available, priced accordingly. The idea of offering segmented tiers (guaranteed power at one price, interruptible power at a lower price, flexible power with real-time dynamic pricing) was not standard practice. But Jensen argued that utilities should see this as an opportunity, not a threat. They could monetize capacity that was currently generating zero revenue. All they had to do was create the contractual and operational frameworks to make it work.

* * *

NVIDIA was already pushing hard on the other side of the equation: making each watt of power go further. Jensen quantified the progress

with characteristic precision. In the ten years preceding the conversation, Moore's Law would have improved computing performance by roughly 100 times. NVIDIA, through extreme co-design (optimizing every layer of the stack simultaneously), had improved its performance by a factor of a million over the same period. The token-generation cost was dropping by an order of magnitude every year. Computer prices were going up, but the value extracted from each unit of power was going up much, much faster.

Still, the appetite for compute was growing faster than even those extraordinary efficiency gains could satisfy. The world needed both more efficient hardware and more available power. Jensen's grid-utilization proposal addressed the second need without requiring a single new power plant, a single new transmission line, or a single new regulatory approval for nuclear or renewable generation (all of which were important but slow). The idle capacity was already there. It was already paid for. It was already connected. Someone just had to use it.

* * *

Beyond the grid, there was a more exotic frontier: space. NVIDIA's GPUs were already orbiting the Earth, processing images from high-resolution satellite cameras. The logic was straightforward. Those satellites generated petabytes of data as they swept the planet, capturing centimeter-scale imagery continuously. Beaming all of that data back to ground stations was impractical. It made far more sense to do the AI processing right there, at the edge, aboard the satellite, discarding everything that was unchanged or uninteresting and transmitting only the novel information.

Jensen acknowledged the engineering challenges. Space offered effectively unlimited solar energy (especially at polar orbits), but it offered almost no cooling. In the vacuum of space, there is no air to

convect heat away, no water to absorb it. The only mechanism left is radiation, which is slow and requires large surface areas. The path to serious compute in orbit would require big, ungainly radiator arrays and entirely new approaches to thermal management.

There were other puzzles: radiation damage, component degradation, the impossibility of physical maintenance, the need for software that could detect and route around failing hardware in real time. Jensen's approach to these challenges was characteristically pragmatic. He sent engineers to work on the problems. They were learning about radiation hardening, graceful degradation, redundancy architectures, and the peculiarities of thermal management in vacuum. The goal was not to build a data center in space next year. It was to cultivate the knowledge so that, when the time came, NVIDIA would be ready.

But Jensen's preferred near-term answer to the energy problem remained firmly terrestrial. Eliminate waste first. That 40 percent of idle grid capacity was sitting right there, on the ground, already built, already generating nothing. Before anyone launched a single rack into orbit, they ought to figure out how to use what was already available on Earth. The low-hanging fruit, as it turned out, was hanging so low that most people had walked right past it without noticing.

CHAPTER 6

The Speed of Light

Why first principles beat continuous improvement, and what Jensen Huang and Elon Musk share in their approach to building impossible things.

* * *

Thirty years ago, before NVIDIA was a household name, before the AI revolution, before the company's market capitalization had even cleared its first billion, Jensen Huang introduced a concept to his engineering teams that would become, over the decades, something close to a corporate religion. He called it "the speed of light."

The phrase did not refer, literally, to the velocity of photons in vacuum. It was shorthand for something broader and more demanding: what is the absolute physical limit of what can be achieved? For any metric that mattered (memory bandwidth, arithmetic throughput, power consumption, latency, manufacturing cycle time, number of engineering hours, cost per unit of performance) Jensen wanted his teams to start by calculating the theoretical maximum, the boundary imposed by the laws of physics, and then measure everything against that standard.

The method sounds simple. In practice, it was a radical departure from how most engineering organizations worked.

* * *

The conventional approach to improving a process is continuous improvement: take what exists, identify the weakest link, and make it a little better. A manufacturing step takes 74 days? Find a way to shave it to 72. The 72-day process is then the new baseline, and you look for ways to get it to 70. Rinse and repeat.

Jensen disliked this method intensely. Not because incremental improvement was useless (it was not), but because it anchored the conversation to the status quo. When you start from 74 days and aim for 72, you have implicitly accepted that 74 is a reasonable starting point. You are optimizing within a frame rather than questioning the frame itself.

Jensen's alternative was to strip the problem back to zero. Why does this process take 74 days? If you were building it from scratch today, with no legacy constraints, no inherited assumptions, no institutional inertia, how long would it take? Often, the answer was shockingly different. Six days. Maybe eight. The gap between six and 74 was not physics. It was accumulated compromises, outdated decisions, organizational habits, and the natural human tendency to accept the familiar as the necessary.

Once you knew that six days was physically possible, the conversation changed fundamentally. You were no longer negotiating a marginal improvement from 74 to 72. You were explaining why each additional day beyond six was a deliberate, justified trade-off. Some of those trade-offs would be legitimate: cost constraints, safety requirements, regulatory compliance. But many would not survive scrutiny. And the ones that did survive would at least be understood, not hidden inside the fog of "that's just how we do it."

* * *

Jensen saw a kindred spirit in Elon Musk, whose construction of the Colossus supercomputer in Memphis had become one of the most talked-about feats of engineering in recent memory. Colossus, a massive AI training cluster powered by 200,000 NVIDIA GPUs, had been built in roughly four months, a timeline that most data-center veterans would have said was physically impossible.

Jensen praised Musk's approach with specific, engineering-level admiration. Musk was a systems thinker who operated across multiple disciplines simultaneously. His method, when confronting any process, was to ask three questions in order: Is this necessary? Does it have to be done this way? Does it have to take this long? By subjecting every element of a project to those questions, Musk stripped away everything that was not essential. What remained was minimal but complete: every component that was necessary for the system to function, and nothing that was not.

Jensen also admired Musk's insistence on being physically present at the point of action. When there was a problem at the Colossus construction site, Musk did not receive a filtered briefing in a conference room. He went to the site, found the engineer doing the work, and said: show me the problem. In one memorable instance, he spent time going through the exact process of plugging cables into a rack, understanding each step at the granular level, looking for ways to make the task less error-prone.

This ground-level engagement built a kind of intuition that no amount of PowerPoint slides could replicate. By understanding the physical reality of every task involved in assembling a data center (from pouring concrete to routing fiber optics to powering up racks), Musk could see inefficiencies at both the micro and macro scales. And he had the organizational authority to do something about it: not just to improve a step, but to eliminate it entirely, or to redesign the system so that the

step was no longer needed.

* * *

The parallels with Jensen's own approach were striking, though the domains differed. Both men operated from first principles. Both distrusted continuous improvement as a primary methodology. Both insisted on understanding the physical limits before accepting any proposed timeline or design. And both were willing to make decisions that the rest of their industries considered impossible, not because they were reckless, but because they had done the math and knew the difference between an actual physical constraint and a conventional assumption dressed up as one.

The difference was one of emphasis. Musk was, by temperament, a minimalist. His instinct was to subtract: remove parts, remove steps, remove overhead. Jensen was, by temperament, a systems integrator. His instinct was to optimize across every dimension simultaneously, finding the combination of trade-offs that produced the best total outcome. Both approaches converged on a common insight: the status quo was almost never the best achievable result, and the gap between current practice and physical limits was almost always larger than anyone expected.

* * *

Jensen distilled his engineering philosophy into a single phrase that he used more than any other: "As complex as necessary, but as simple as possible." The Vera Rubin pod, with its seven chip types, five rack types, 40 racks, and 1.2 quadrillion transistors, was undeniably complex. But was all that complexity necessary? That was the question Jensen wanted his teams to ask, relentlessly, about every component and every design decision. If the answer was yes, the complexity stayed. If the

answer was no, or even maybe, it was challenged, scrutinized, and, if possible, eliminated.

The result was systems that looked, from the outside, impossibly intricate. But viewed from the inside, with an understanding of the physical limits and the engineering trade-offs, each element existed for a reason. Nothing was gratuitous. The complexity was the minimum required to solve the problem.

Jensen called it the most complex computer the world had ever made. He said it with pride, but also with the quiet confidence of someone who knew that every piece of that complexity had been tested against the speed of light and found to be necessary.

CHAPTER 7

The Builder Nation

How China became the fastest-innovating country in the world, and what open source has to do with it.

* * *

Roughly half of the world's AI researchers are Chinese. The figure is approximate ("plus or minus," as Jensen put it), but even if it were off by ten points in either direction, the implication would be the same. The most consequential technology of the twenty-first century was being developed, in large part, by a talent pool rooted in a single country. Some of those researchers worked abroad, at American universities and Silicon Valley labs. But a great many of them remained in China, embedded in a domestic technology ecosystem that, over the preceding decade, had undergone one of the most rapid and remarkable transformations in industrial history.

Jensen Huang, who was born in Taiwan and maintained deep relationships across East Asia, had a nuanced and often surprising perspective on how China had achieved this position. His explanation did not rely on government subsidies or strategic planning (though those played a role). It was, at its core, a story about culture, timing, competition, and the peculiar sociology of open-source software.

* * *

The timing mattered enormously. China's technology industry had come of age during the mobile-cloud era, which meant that its foundational skillset was software, not hardware. The country had always produced formidable mathematicians and scientists. When those minds were channeled into a technology economy built around apps, cloud services, and digital platforms, the result was an extraordinarily software-literate workforce. Unlike earlier generations of Chinese engineers, who had worked primarily in manufacturing and heavy industry, this generation was comfortable with modern development tools, agile methodologies, and the rapid iteration cycles of internet-era software.

China was also, Jensen observed, not a single monolithic economy. It was a collection of provinces and cities, each with its own ambitious local government, each competing fiercely with the others for economic prestige. The result was a kind of Darwinian duplicator: whatever technology or industry was emerging, every province wanted its own version. Electric vehicles? Dozens of Chinese EV companies sprang up. AI? Dozens of AI companies. Robotics? Cloud computing? Enterprise software? Dozens of each. The competition was intense, sometimes wasteful, and occasionally chaotic. But what emerged from the other end of that Darwinian funnel was, almost without exception, extraordinary.

* * *

Then there was the cultural dimension, which Jensen described with a warmth that suggested personal experience. Chinese society organized itself around a hierarchy of loyalty: family first, friends second, company third. The practical consequence for the technology sector was that knowledge flowed with remarkable freedom. An engineer at one company might have a brother at a competitor. Their schoolmates might be scattered across a dozen firms. The concept of the "schoolmate bond"

(a lifelong loyalty forged in university) meant that information and insights circulated through informal networks far faster than they could through any official channel.

This, Jensen argued, was the real explanation for China's outsized contribution to open-source software. It was not that Chinese companies had made a strategic decision to embrace open source. It was that the culture was already open source, in every way that mattered. Engineers shared knowledge reflexively, because their social bonds cut across corporate boundaries. Keeping technology hidden from people who were, in the most literal sense, your brothers and schoolmates felt unnatural. You might as well publish it, formalize it, and let the whole community benefit.

The open-source culture then amplified everything else. Rapid knowledge transfer accelerated innovation. Innovation attracted more talent. More talent produced more open-source contributions. The flywheel spun faster and faster, driven by forces that were fundamentally cultural rather than strategic.

* * *

Jensen drew an explicit contrast with the United States. American leaders, he noted, were mostly lawyers, products of a society organized around the rule of law and the protection of individual rights. Chinese leaders (particularly at the provincial and city level) were more likely to be engineers, products of a society that had been built, within living memory, out of poverty. The American system had its own profound strengths, but it did not, as a cultural matter, celebrate the engineer in the same way.

In China, it was cool to be a builder. The social status of engineering was high. Parents wanted their children to study math and science. The education system, whatever its other shortcomings, produced students

who were formidably well-prepared in technical disciplines. All of these factors, operating together over the span of a single generation, had produced what Jensen described simply as a builder nation.

* * *

NVIDIA's response to this reality was not to compete against Chinese innovation but to amplify it. The company's open-source AI model, Nemotron 3 Super, was released with its models, weights, training data, and methodology all publicly available. It was a true open-source release, not the partial, strategically hedged variety that some companies offered.

Jensen's reasoning was characteristically systematic. First, NVIDIA needed to understand how AI models were evolving in order to design the hardware that would run them. Building and training its own models gave the company firsthand insight into the computational demands of cutting-edge architectures. Nemotron 3, which combined transformer and state-space model architectures, was not just a product. It was a research instrument, a tool for co-designing the future of computing hardware.

Second, NVIDIA recognized that AI would only achieve its full potential if it diffused into every industry, every country, and every research lab on Earth. Proprietary models were essential for some applications, but they were insufficient for the kind of broad-based adoption that would transform entire economies. Open-source models were the seeds that would plant AI in soil that proprietary companies could not reach.

Third, and most ambitiously, Jensen saw that AI was not just a language technology. Future AI systems would need specialized sub-models trained on biology, chemistry, physics, fluid dynamics, materials science, and dozens of other domains that did not fit neatly

into the text-in, text-out paradigm of large language models. Someone had to push the frontier in all of those areas. NVIDIA did not build cars or discover drugs, but it could make sure that every car company and every pharmaceutical company had access to world-class models for their domain.

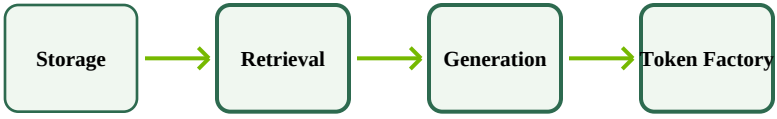
The open-source strategy was, in the end, another expression of the same philosophy that governed everything Jensen did: inform, share, inspire, and let the ecosystem do what only an ecosystem can do. You could not build the AI revolution alone. You could only build the platform on which others would build.

CHAPTER 8

The Token Factories

Why the computer is no longer a warehouse, the data center is no longer a cost center, and NVIDIA's future is limited only by the world's imagination.

From Warehouse to Factory: The Computing Paradigm Shift



Computing evolved from file retrieval to real-time intelligence generation.

* * *

For most of the history of computing, a data center was, functionally, a very expensive filing cabinet. Humans created content (documents, images, videos, web pages, database entries) and stored it on servers. When someone needed that content, the system retrieved it, applied some filtering or recommendation logic, and delivered the right file at the right time. The entire architecture, from the storage arrays to the networking to the software, was optimized for a single task: retrieval.

Jensen Huang had a blunter way of putting it. The old data center was a warehouse. It stored things. It did not make things. And

warehouses, as anyone in business knows, do not generate much revenue. They are cost centers, necessary but unglamorous, justified only by the value of the goods they hold.

The AI revolution had turned the warehouse into a factory.

* * *

The transformation was not a matter of degree. It was a change in kind. AI computing did not retrieve pre-made answers from storage. It generated new, contextually relevant, situation-aware responses in real time. Every query required processing. Every response required computation. The data center was no longer fetching files; it was manufacturing intelligence, token by token, on demand.

This distinction mattered enormously for the economics of the industry. A warehouse is valued by its capacity: how many files it can store, how fast it can retrieve them. A factory is valued by its output: how many products it can make, how fast, and at what quality. The metrics are completely different. And the relationship between investment and revenue is completely different. A warehouse is a sunk cost. A factory is a revenue engine. Every dollar spent improving a factory's throughput translates, directly, into more products to sell.

Jensen saw the tokens produced by AI factories not as an undifferentiated commodity but as a segmented product line, analogous to how Apple had segmented the smartphone market. There were free tokens, the everyday outputs of basic chat and search. There were premium tokens, the results of deep reasoning, specialized analysis, or domain-specific expertise. And there were tiers in between. Intelligence, it turned out, was a scalable product. Some tokens were worth almost nothing. Others, the ones that could solve a complex legal problem, design a novel molecule, or debug a critical piece of infrastructure, might be worth a thousand dollars per million. Jensen did not see this as

a speculative possibility. He saw it as an inevitability, a question of when, not if.

* * *

This was the foundation of Jensen's argument for why NVIDIA's growth was, in his word, "inevitable." The argument rested on two shifts, both of which he considered irreversible.

The first shift was computational. The world had moved from retrieval-based computing (which needed lots of storage and little processing) to generative computing (which needed enormous amounts of processing and a fundamentally different hardware architecture). The only scenario in which this trend would reverse was if generative AI turned out to be ineffective, if, after all the investment and all the hype, the technology simply did not work. Jensen found this scenario vanishingly unlikely. Each passing year had given him more confidence, not less. The last five years, in particular, had been more convincing than the ten that preceded them.

The second shift was economic. Computers had gone from being cost centers (warehouses that stored information) to profit centers (factories that generated revenue). This meant that the percentage of global GDP devoted to computing was going to increase, potentially by a factor of a hundred or more. Not because companies were becoming more profligate with their IT budgets, but because the computer itself had become a productive asset, a machine that made things people would pay for.

* * *

NVIDIA's moat, the term Silicon Valley used for a company's competitive defense, was not any single technology. It was the install base.

Jensen was explicit about this. CUDA, the computing platform that NVIDIA had nearly bankrupted itself to create, had accumulated millions of developers, hundreds of millions of deployed GPUs, and a software ecosystem that spanned every cloud, every computer company, every industry, and every country on Earth. A developer who wrote software for CUDA could reach an audience that no other computing platform could match. The software would run on machines in Google's data centers, in Amazon's cloud, in enterprise servers, in autonomous vehicles, in robots, in satellites. That ubiquity was not an accident. It was the result of twenty years of sustained investment, and it was, in Jensen's assessment, NVIDIA's single most important asset.

The velocity of NVIDIA's execution amplified the advantage. No company in history had built systems of NVIDIA's complexity, and certainly none had done so on a one-year product cycle. From a developer's perspective, this meant that CUDA would be ten times more powerful in six months. The platform was not just large; it was improving at a rate that made alternatives look like standing still. And the trust factor was decisive. Developers believed that NVIDIA would continue to maintain, improve, and invest in CUDA for as long as the company existed. That belief was worth more than any technical specification.

* * *

Jensen's mental model of what NVIDIA built had evolved dramatically over the decades. In the early days, he visualized a chip. When announcing a new product, he would hold up the silicon die, small enough to fit in his palm, and present it as the star of the show. That era was over. The chip was, as he put it with affection, "still adorable." But it was no longer the mental model.

The mental model now was a gigawatt-scale AI factory connected to the power grid, cooled by industrial systems, networked with unimaginable bandwidth, staffed by thousands of engineers, and producing tokens at a rate that would have been inconceivable a few years earlier. Powering up one of these factories was not a matter of flipping a switch. It took thousands of people working in concert over weeks or months to bring the system online.

Jensen's ambition did not stop at individual factories. His next "click," as he called it, was planetary scale: a network of AI factories spanning the globe, interconnected, load-balanced, and operating as a single computational entity. And beyond that, though he was characteristically pragmatic about timelines, lay the frontier of space-based computation, where the laws of physics offered both extraordinary opportunities (unlimited solar energy) and extraordinary challenges (no air, no water, only radiation for cooling).

* * *

The question that hung over all of it, the question that investors and journalists and competitors could not stop asking, was: how big could this get? Could NVIDIA be worth ten trillion dollars?

Jensen's answer was disarmingly direct. Every supposed limit that had been placed on NVIDIA's growth had turned out to be wrong. A CEO had once told him it was theoretically impossible for a fabless semiconductor company to exceed a billion dollars in revenue. Others had insisted that NVIDIA would never surpass 25 billion. Each prediction had been based not on physical constraints but on a failure of imagination, an inability to see beyond the current paradigm.

NVIDIA, Jensen pointed out, was not in the market-share business. The company was not trying to take a slice of an existing pie. Almost everything it was building was new. There was no incumbent to

displace, no existing market to carve up. The opportunity had to be imagined into existence, and then built, piece by piece, from the ground up.

That was the hardest part. Not the engineering. Not the supply chain. Not the power grid. The hardest part was getting the world to imagine a future that did not yet exist, and then to invest in building it before the returns were visible. Jensen had been doing exactly that for thirty years. He saw no reason to stop.